

VQuery User Guide

Version: 1.0.4

Author: Ouerm Davy

E-mail: ravyouerm89@gmail.com

Table of Contents

1. Introduction
 2. Features
 3. Installation
 4. Configuration
 5. MySQL Connection
 6. PostgreSQL Connection
 7. SQL Server Connection
 8. CRUD Operations
 9. Parameterized Queries
 10. Transactions
 11. ExecuteScalar
 12. ExecuteNonQuery
 13. Query
 14. QueryFirst
 15. QuerySingle
 16. QueryFirstOrDefault
 17. QuerySingleOrDefault
 18. Exists
 19. Count
 20. Async Methods
 21. QueryMultiple
 22. Bulk Insert
 23. ReflectionCache
 24. Performance Tips
 25. Best Practices
 26. Error Handling
 27. Complete Examples
-

Introduction

VQuery is a lightweight ORM and database helper library for .NET.

Supported databases:

- MySQL
- PostgreSQL
- SQL Server

VQuery provides:

- Fast object mapping
 - Async support
 - Transactions
 - Parameterized queries
 - ReflectionCache optimization
 - Bulk operations
-

Installation

Install from NuGet:

```
dotnet add package VQuery
```

Or Package Manager:

[Install-Package VQuery](#)

Configuration

MySQL Configuration

```
"MSQLConnection": {  
  "server": "127.0.0.1",  
  "port": "3306",  
  "database": "sampledb",  
  "username": "root",  
  "password": "123456",  
  
  "pooling": "true",  
  "minimumPoolSize": "5",  
  "maximumPoolSize": "100",  
  
  "connectionTimeout": "30",  
  "commandTimeout": "60"  
}
```

Setting	Description	Default
server	MySQL server hostname or IP address	127.0.0.1
port	MySQL port number	3306

Setting	Description	Default
database	Database name	sampledb
username	Database username	root
password	Database password	123456
pooling	Enable connection pooling	true
minimumPoolSize	Minimum pooled connections	5
maximumPoolSize	Maximum pooled connections	100
connectionTimeout	Connection timeout (seconds)	30
commandTimeout	Command timeout (seconds)	60

PostgreSQL Configuration

```
"POSTGREConnection": {
  "server": "127.0.0.1",
  "port": "5432",
  "database": "sampledb",
  "username": "postgres",
  "password": "123456",

  "pooling": "true",
  "minimumPoolSize": "5",
  "maximumPoolSize": "100",

  "connectionTimeout": "30",
  "commandTimeout": "60"
}
```

Setting	Description	Default
server	PostgreSQL server hostname or IP address	127.0.0.1
port	PostgreSQL port number	5432
database	Database name	sampledb
username	Database username	postgres
password	Database password	123456
pooling	Enable connection pooling	true
minimumPoolSize	Minimum pooled connections	5
maximumPoolSize	Maximum pooled connections	100
connectionTimeout	Connection timeout (seconds)	30
commandTimeout	Command timeout (seconds)	60

SQL Server Configuration

```
"SQLSERVERConnection": {
  "server": "127.0.0.1",
  "port": "1433",
  "database": "sampledb",
```

```

"username": "sa",
"password": "123456",

"pooling": "true",
"minimumPoolSize": "5",
"maximumPoolSize": "100",

"connectionTimeout": "30",
"commandTimeout": "60",

"trustServerCertificate": "true"
}

```

Setting	Description	Default
server	SQL Server hostname or IP address	127.0.0.1
port	SQL Server port number	1433
database	Database name	sampledb
username	Database username	sa
password	Database password	123456
pooling	Enable connection pooling	true
minimumPoolSize	Minimum pooled connections	5
maximumPoolSize	Maximum pooled connections	100
connectionTimeout	Connection timeout (seconds)	30
commandTimeout	Command timeout (seconds)	60
trustServerCertificate	Trust SQL Server SSL certificate	true

Complete appsettings.json Example

```

{
  "ConnectionStrings": {

    "MYSQLConnection": {
      "server": "127.0.0.1",
      "port": "3306",
      "database": "sampledb",
      "username": "root",
      "password": "123456",
      "pooling": "true",
      "minimumPoolSize": "5",
      "maximumPoolSize": "100",
      "connectionTimeout": "30",
      "commandTimeout": "60"
    },

    "POSTGREConnection": {
      "server": "127.0.0.1",
      "port": "5432",
      "database": "sampledb",

```

```

        "username": "postgres",
        "password": "123456",
        "pooling": "true",
        "minimumPoolSize": "5",
        "maximumPoolSize": "100",
        "connectionTimeout": "30",
        "commandTimeout": "60"
    },

    "SQLSERVERConnection": {
        "server": "127.0.0.1",
        "port": "1433",
        "database": "sampledb",
        "username": "sa",
        "password": "123456",
        "pooling": "true",
        "minimumPoolSize": "5",
        "maximumPoolSize": "100",
        "connectionTimeout": "30",
        "commandTimeout": "60",
        "trustServerCertificate": "true"
    }
}
}
}

```

Opening Connections

MySQL

```
using var db = new MySqlConnection();
```

```
db.ConnectionOpen();
```

PostgreSQL

```
using var db = new PostgreSQLConnection();
```

```
db.ConnectionOpen();
```

SQL Server

```
using var db = new SQLServerConnection();
```

```
db.ConnectionOpen();
```

Closing Connections

```
db.ConnectionClose();
```

Recommended:

```
using var db =  
    new MySqlConnection();  
  
db.ConnectionOpen();  
  
/* database work */
```

The connection is automatically closed when disposed.

Connection Status

```
if (db.IsConnected)  
{  
    Console.WriteLine("Connected");  
}
```

SelectDataTable

```
DataTable dt =  
    db.SelectDataTable(  
        "SELECT * FROM users");
```

SelectDataRow

```
DataRow row =  
    db.SelectDataRow(  
        "SELECT * FROM users WHERE id=1");
```

SelectDataValue

```
string name =  
    db.SelectDataValue(  
        "SELECT name FROM users WHERE id=1");
```

SelectDataScalar

Returns the first column of the first row as a string.

Example:

string total =

```
db.SelectDataScalar(  
    "SELECT COUNT(*) FROM users");
```

Parameterized Example:

string name =

```
db.SelectDataScalar(  
    "SELECT name FROM users WHERE id=@id",  
    new()  
    {  
        ["@id"] = 1  
    });
```

Insert

Raw SQL

```
db.Insert(  
    "INSERT INTO users(name) VALUES('John')");
```

Parameterized

```
db.Insert(  
    "INSERT INTO users(name) VALUES(@name)",  
    new()  
    {  
        ["@name"] = "John"  
    });
```

Update

```
db.Update(  
    "UPDATE users SET name=@name WHERE id=@id",  
    new()  
    {
```

```
        ["@name"] = "David",
        ["@id"] = 1
    });
```

Delete

```
db.Delete(
    "DELETE FROM users WHERE id=@id",
    new()
    {
        ["@id"] = 1
    }
);
```

Transactions

```
db.Begin();

try
{
    db.Insert(...);
    db.Update(...);

    db.Commit();
}
catch
{
    db.Rollback();
}
```

Execute

Executes SQL and returns true when successful.

Example:

```
bool success =
    db.Execute(
        "UPDATE users SET active=1");
```

Parameterized Example:

```
bool success =
    db.Execute(
        "UPDATE users SET active=@active WHERE id=@id",
```



```
new()
{
    ["@active"] = 1,
    ["@id"] = 10
});
```

ExecuteRun

Executes SQL and returns the execution result.

Example:

```
bool result =
    db.ExecuteRun(
        "CALL sp_update_customer()");
```

Parameterized Example:

```
bool result =
    db.ExecuteRun(
        "CALL sp_update_customer(@id)",
        new()
        {
            ["@id"] = 100
        });
```

ExecuteAsync

Asynchronously executes SQL.

Example:

```
bool success =
    await db.ExecuteAsync(
        "UPDATE users SET active=1");
```

Parameterized Example:

```
bool success =
    await db.ExecuteAsync(
        "UPDATE users SET active=@active WHERE id=@id",
        new()
        {
```

```
        ["@active"] = 1,  
        ["@id"] = 1  
    });  
  


---


```

ExecuteScalar

```
int total =  
    db.ExecuteScalar<int>( "SELECT COUNT(*) FROM users");  
  


---


```

ExecuteScalarAsync

```
int total =  
    await db.ExecuteScalarAsync<int>( "SELECT COUNT(*) FROM users");  
  


---


```

ExecuteNonQuery

```
int rows =  
    db.ExecuteNonQuery( "UPDATE users SET active=1");  
  


---


```

Query

Model:

```
public class User  
{  
    public int Id { get; set; }  
  
    public string Name { get; set; }  
}
```

Query:

```
var users =  
    db.Query<User>( "SELECT * FROM users");
```

QueryAsync

```
var users =  
    await db.QueryAsync<User>(  
        "SELECT * FROM users");
```

QueryFirst

```
var user =  
    db.QueryFirst<User>(  
        "SELECT * FROM users");
```

QuerySingle

```
var user =  
    db.QuerySingle<User>(  
        "SELECT * FROM users WHERE id=1");
```

Throws exception if:

- No record found
- More than one record found

QueryFirstOrDefault

```
var user =  
    db.QueryFirstOrDefault<User>(  
        "SELECT * FROM users");
```

QuerySingleOrDefault

```
var user =  
    db.QuerySingleOrDefault<User>(  
        "SELECT * FROM users WHERE id=1");
```

Returns null when no record exists.

Exists

```
bool exists =
    db.Exists(
        "SELECT COUNT(*) FROM users WHERE id=@id",
        new()
        {
            ["@id"] = 1
        }
    );
```

ExistsAsync

```
bool exists =
    await db.ExistsAsync(
        "SELECT COUNT(*) FROM users WHERE id=@id",
        new()
        {
            ["@id"] = 1
        }
    );
```

Count

```
int total =
    db.Count(
        "SELECT COUNT(*) FROM users");
```

CountAsync

```
int total =
    await db.CountAsync(
        "SELECT COUNT(*) FROM users");
```

QueryMultiple

```
string sql =
    @"
    SELECT * FROM users;
    SELECT * FROM roles;
    ";

var result =
    db.QueryMultiple(sql);
```

```
DataTable users =
```

```
result.Tables[0];
```

```
DataTable roles =  
    result.Tables[1];
```

Bulk Insert

```
db.InsertMultiple(customers);
```

ReflectionCache

VQuery automatically caches model property metadata to improve ORM mapping performance.

Benefits:

- Faster Query()
- Faster QueryAsync()
- Reduced reflection overhead
- Lower CPU usage
- Better scalability

ReflectionCache is automatically used by:

- Query()
- QueryAsync()
- QueryFirst()
- QuerySingle()
- QueryMapper
- ReaderMapper

No configuration is required.

Error Handling

```
try  
{  
    db.ConnectionOpen();  
}  
catch(Exception ex)  
{  
    Console.WriteLine(ex.Message);  
}
```

Best Practices

1. Always use parameterized queries.
 2. Use QueryAsync in ASP.NET Core.
 3. Use QuerySingle when exactly one row is expected.
 4. Use QueryFirst when multiple rows may exist.
 5. Use transactions for multiple updates.
 6. Dispose connections using using statement.
 7. Avoid SELECT * in production systems.
-

Performance Tips

Recommended:

- Enable connection pooling.
 - Use QueryAsync for web applications.
 - Use ReflectionCache.
 - Reuse connections.
 - Use parameterized SQL.
-

Complete Example

```
using VQuery;

using var db =
    new MySqlConnection();

db.ConnectionOpen();

var users =
    await db.QueryAsync<User>(
        "SELECT * FROM users WHERE active=@active",
        new()
        {
            ["@active"] = 1
        }
    );

foreach(var user in users)
{
    Console.WriteLine(
        user.Name);
}
```

VariableConverter Functions

ToInt

```
int value =  
    db.ToInt("100");
```

ToDouble

```
double value =  
    db.ToDouble("123.45");
```

ToDate

```
DateTime date =  
    db.ToDate(  
        "01-01-2026");
```

ToDateTime

```
DateTime dateTime =  
    db.ToDateTime(  
        "01-01-2026 12:30:00");
```

NumberToText

```
string text =  
    db.NumberToText(  
        1000);
```

Output:

One Thousand

NumberToTextKH

```
string text =  
    db.NumberToTextKH(  
        1000);
```

NumberToKhNumber

```
string value =  
    db.NumberToKhNumber(  
        12345);
```

End of Document

Additional Documentation Updates (V1.1.0)

Provider Support Matrix

Feature	MySQL	PostgreSQL	SQL Server
Connection Open	Yes	Yes	Yes
Connection Pooling	Yes	Yes	Yes
CRUD Operations	Yes	Yes	Yes
Transactions	Yes	Yes	Yes
Execute	Yes	Yes	Yes
ExecuteRun	Yes	Yes	Yes
ExecuteScalar<T>	Yes	Yes	Yes
ExecuteNonQuery	Yes	Yes	Yes
Query<T>	Yes	Yes	Yes
QueryMultiple	Yes	Yes	Yes
Async Support	Yes	Yes	Yes

Generic ORM Mapping

VQuery automatically maps database columns to model properties using ReflectionCache.

ReflectionCache

Property metadata is cached and reused to improve Query<T>() and QueryAsync<T>() performance.

Missing Async Examples

ExecuteNonQueryAsync(), QuerySingleAsync(), QueryFirstOrDefaultAsync(), QuerySingleOrDefaultAsync(), ExistsAsync().

Connection Pooling

Recommended: minimumPoolSize=5, maximumPoolSize=100, connectionTimeout=30, commandTimeout=60.

Final Documentation Improvements

Additional Async Methods

```
QueryFirstAsync<T>()  
QuerySingleAsync<T>()  
QueryFirstOrDefaultAsync<T>()  
QuerySingleOrDefaultAsync<T>()  
ExecuteNonQueryAsync()
```

Example:

```
var user = await db.QueryFirstAsync<User>("SELECT * FROM users");
```

Complete VariableConverter Functions

```
ToString()  
ToStringDate()  
ToStringDateTime()  
ToIntString()  
ToDoubleString()  
IsEmpty(DataRow)
```

Examples:

```
string date = db.ToStringDate(DateTime.Now);  
bool empty = db.IsEmpty(row);
```

Generic ORM Mapping

VQuery automatically maps database columns to model properties.

Example:

```
ID -> Id  
NAME -> Name  
EMAIL -> Email
```

The mapping is case-insensitive and optimized using ReflectionCache.

NuGet

NuGet Package:

<https://www.nuget.org/packages/VQuery>

Version History

V1.0.4

Features:

- MySQL Support
- PostgreSQL Support
- SQL Server Support
- Async Support
- ReflectionCache
- QueryMultiple
- Bulk Insert
- Connection Pooling